

Final Project

Tiny Neural Network Accelerator

Prof. 문 승 현

Date: 2025. 06. 23.

전기전자공학부

202011021

장 시 영

1 Design Architecture

1) Overall Architecture

해당 프로젝트에서는 이전까지 구현한 systolic array 모듈을 FC layer로 활용하여 두 개의 Fully Connected (FC) layer, 1개의 Normalization layer, 1개의 ReLU layer로 구성된 Tiny Neural Network Accelerator 구현을 목표로 한다.

FC layer에서는 8x8 연산을 위해 tiling 기법을 활용하며 전체 네트워크 구조는 다음과 같다.

- FC1: 8x8 weight matrix(W1), 입력(X) 크기 8x8(base) or 8x16(extra)
- Norm: 각 FC1의 출력값을 32로 나눔
- ReLU: 음수 값을 0으로 변경
- FC2: 8x8 weight matrix(W2), 입력은 Norm -> ReLU layer를 통과한 결과값
- 출력(Y): 8x8 or 8x16, 16bit

1-1) Top-level interface

Top module, 즉 nn_accelerator 모듈의 주요 포트 및 설명은 다음과 같다.

Range	Port	Bitwidth	Direction	Operation
System	run	1	Input	연산 시작 알림
	batch_mode	1	Input	0: 8-batch, 1: 16-batch
State	State	4	Output	FSM state
	busy	1	Output	State가 동작 중인 경우
Input Vector	re_X	1	Output	Memory X read-enable
	addr_X	7	Output	8bit data 주소
	data_X	8	Input	Memory X에서 읽은 데이터
FC1 weight	re_W1	1	Output	W1 read-enable
	addr_W1	6	Output	Weight 파일 내 주소
	data_W1	8	Input	W1 데이터
FC2 weight	re_W2	1	Output	W1 read-enable
	addr_W2	6	Output	Weight 파일 내 주소
	data_W2	8	Input	W1 데이터
Output Vector	re_Y_ext	1	Input	외부에서 Y 값 읽기 요청
	addr_Y_ext	7	Input	외부에서 읽고 싶은 Y 데이터 주소
	data_Y	16	Output	Memory1에서 읽은 16-bit 결과

1-2) Inner module interface

또한, nn_accelerator 내부에서 활용하는 모듈들의 인터페이스 및 해당 모듈들의 동작은 다음과 같다.

Module	Interface	Operation
Matmul_sa U_FC1	run, state, re_A/B	4x4 output-stationary SA A = W1(8bit), B = X(8bit) 8x8 연산 후 16bit 결과 출력
	addr_A/B, data_A/B	
	we_C, addr_C, data_C	
Norm_layer	en, din, dout, valid	5bit shift
Relu_layer	en, din, dout, valid	MSB가 1인 경우 0 0인 경우 dout = din
Matmul_sa U_FC2	run, state, re_A/B	U_FC1과 동일
	addr_A/B, data_A/B	
	we_C, addr_C, data_C	

1-3) FSM

Nn_accelerator에서 동작하는 state와 해당되는 FSM의 구성 및 동작은 다음과 같다. signal에는 해당 state에서 제어하는 주요 신호들이다.

State	Signal	Operation
IDLE	-	대기
FC1_EXEC	fc1_run, we_mem1	FC1 실행 & mem x1 기록
NORM_EXEC	norm_en, we_mem2	mem x1 -> Norm -> mem x2
RELU_EXEC	relu_en, we_mem3	mem x2 -> ReLU -> mem x3
FC2_EXEC	fc2_run, we_mem1	FC2 실행 & mem x1에 Y 기록
WRITE_OUT	re_mem1	mem x1의 최종 결과 출력
DONE	-	

1-4) Overall clock flow

해당 모듈의 state, 그에 해당하는 clock 및 세부 clock 구성은 다음과 같다.

State	Clock cycle count	Detailed clock cycle
IDLE	-	
FC1_EXEC	0 – 605	MAC operation 20 – 67, 87 – 134, 171 – 218, 238 – 285, 322 – 369, 389 – 436, 473 – 520, 540 – 587 Data write out 135 – 151, 286 – 302, 437 – 453, 588 – 604
NORM_EXEC	606 – 674	
RELU_EXEC	675 – 743	
FC2_EXEC	744 – 1349	MAC operation 764 – 811, 831 – 878, 915 – 962, 982 – 1029, 1066 – 1113, 1133 – 1180, 1217 – 1264, 1284 – 1331 Data write out 879 – 895, 1030 – 1046, 1181 – 1197, 1332 – 1348
WRITE_OUT	1350 – 1418	
DONE	-	

즉, 해당 모듈에서는 다음과 같은 순서로 neural network 연산을 진행한다.

첫번째, 텍스트 파일로 주어진 X와 W1을 불러와 FC1 모듈에서 systolic array 연산을 진행한다. 이후 연산 진행 결과는 memory X1에 저장하도록 한다.

두번째, X1으로부터 데이터를 불러와 Normalization을 통해 32로 나누며 해당 결과를 memory x2에 저장하도록 한다.

세번째, memory x2에서 normalization 연산 결과를 가져와 relu 연산을 진행하며 해당 결과를 memory x3에 저장하도록 한다.

네번째, memory x3에 저장된 데이터를 불러와 FC2 모듈에서 input으로 사용하며 텍스트 파일로 제공된 W2를 호출하여 systolic array 연산을 진행한다. 해당 결과는 별도의 메모리를 추가로 선언하지 않고 기존에 존재하던 memory x1에 저장하도록 한다.

마지막으로 memory x1에 접근하여 최종 결과를 출력한다.

위 과정은 base(8-batch), extra(16-batch)에 동일하게 적용되지만, memory에 접근하여 데이터를 작성하거나 출력하는 경우 각 배치 크기에 맞게 주소 인덱싱을 달리하도록 한다.

2) Questions

현재 코드에서는 한 cycle의 주기를 10ns로 하였으므로 clock frequency는 100MHZ이다.

2-1) What is the peak bandwidth of my design in [MB/s]

- Peak bandwidth를 nn_accelerator 모듈 기준으로 보도록 한다.
- Input port의 최대는 8bit이며, output port의 최대는 16bit이다.
- 따라서, 이 중 최대값인 16bit를 peak bandwidth 기준으로 한다.
- 그렇다면 2byte/clock이 가능해지며 이때 peak bandwidth는 200MB/s이다.

2-2) What is the peak performance of my design in [GOPS]

- 현재 디자인에서는 FC1 -> Norm -> ReLU -> FC2 라는 순서 관계가 명확한 형태이다.
- 따라서, peak performance를 측정할 때 활용하는 FC layer의 경우 하나만 사용해야 한다.
- 만약 FC1과 FC2가 동시 동작이나 파이프라이닝이 가능한 형태라면 총 32개의 se를 peak performance에 반영해야 한다.
- 하지만 현재 디자인에서는 하나의 FC layer 즉, 16개의 se만을 고려한다. 또한, 4cycle당 한 번의 MAC 연산이 수행되므로 기존 GOPS에서 4를 나누도록 한다.
- $32Ops(16MAC * 2OPs)/clock * 100Mclock = 3.2GOPS$
- $3.2GOPS / 4cycle \rightarrow 0.8GOPS$

2-3) Estimate the overall latency of my design in [us]

- 8-batch: 13.82us
- 16-batch: 28.01us

2-4) Estimate the utilization ratio of my design in [%]

- 8-batch의 경우, FC1과 FC2 layer에서 적어도 하나의 실제 MAC 연산이 적용되는 cycle은 762 cycle이다.
- 이때, utilization = $7.62us/13.82us$ 이며, utilization ratio = 약 55%이다.
- 16-batch의 경우, FC1과 FC2 layer에서 적어도 하나의 실제 MAC 연산이 적용되는 cycle은 8-batch의 정확히 2배이며 1514 cycle이다.
- 이때, utilization = $15.14us/28.01us$ 이며, utilization ratio = 약 54%이다.

2-5) Discuss how the overall latency and utilization ratio would change if the peak bandwidth of my design were doubled and halved

- 만약 peak bandwidth를 top-module인 nn_accelerator의 16bit 출력 port 기준으로 한다면, peak bandwidth의 변화는 latency에 영향을 미치지 않는다.
- 하지만, peak bandwidth를 모듈간 인터페이스의 데이터 크기로 정의하게 된다면 latency와 utilization ratio는 변경된다.
- 현재 fc1에서 8bit 데이터를 메모리에서 가져오는 데 총 136, 메모리에 16bit결과를 입력하는데 64 cycle이 소모된다. 이는 fc2에서도 동일하다. 메모리 영역을 사용하는 Norm과 ReLU는 모두 각각 64 cycle을 사용하지만, peak bandwidth만을 고려한다면 fc layer에서의 16bit 결과 저장 cycle만을 고려한다.
- 즉, 만약 FC layer 2개에서 16bit 64cycle에 대해 BW를 2배로 한다면 32bit 32cycle로 변경될 것이고 BW를 1/2배로 한다면 8bit 128cycle이 될 것이다.
- 또한 MAC 연산은 8-batch에서 762, 16-batch에서 1514 cycle 고정임을 주의하자.

항목 (8-batch)	Default	BW 2x	BW 1/2
Latency(us)	13.82	$13.82 - .32(64 \rightarrow 32) = 13.50$	$13.82 + .64(64 \rightarrow 128) = 14.46$
Utilization(%)	55	56.4	0.527

항목 (16-batch)	Default	BW 2x	BW 1/2
Latency(us)	28.01	$28.01 - .64(128 \rightarrow 64) = 27.37$	$28.01 + 1.28(128 \rightarrow 256) = 29.29$
Utilization(%)	54	55.3	0.519

즉, 모듈에서 데이터를 주고 받는 bandwidth에 대해 peak bandwidth가 증가하는 경우 utilization이 증가하고, peak bandwidth가 감소하는 경우 utilization이 감소함을 확인할 수 있다.

2-6) Discuss how the overall latency and utilization ratio would change if the systolic array size of my design were doubled and halved

- 8-batch를 기준으로, 기존 디자인에서는 fc1 연산에 $(47 + 17) \times 8 + (16 \times 4)$ 만큼의 cycle이 필요하다.
- 이는 8x8를 4x4 tiling 연산으로 진행하기 때문이며 만약 systolic array 크기가 2배가 되는 경우 tiling을 진행하지 않기에 (총 SE x MAC cycle + 마지막 SE 기준 MAC 연산 + 데이터 출력 + 데이터 입력) x 연산 반복 횟수 + 결과 저장 포맷으로 작성한다면 $(13 \times 4 + 7 \times 4 + 8 + 64) \times 1 + 64 \rightarrow 216$ cycle이 소모된다.
- Systolic array 크기가 1/2배라면, $(3 \times 4 + 1 \times 4 + 2 + 4) \times 64 + (4 \times 16) \rightarrow 1472$ cycle이 소모된다.
- 또한, systolic array 크기 변경에 따른 연산량 감소는 기본 systolic array에서 타이밍을 추가한 현재 타이밍이 아닌, $(44 + 16) \times 8 + (16 \times 4) \rightarrow 544$ cycle을 기준으로 해야한다.

항목 (8-batch)	Default	SA 2x	SA 1/2
Latency(us)	13.82	$13.82 - 6.56(544 \times 2 \rightarrow 216 \times 2) = 7.26$	$13.82 + 18.56(544 \times 2 \rightarrow 1472 \times 2) = 32.38$
Utilization(%)	55	24.1 (새로운 연산 횟수 / latency) $= (87 \times 2 / 726)$	75.0 (새로운 연산 횟수 / latency) $= (1216 \times 2 / 3238)$

항목 (16-batch)	Default	SA 2x	SA 1/2
Latency(us)	28.01	$28.01 - 13.12(544 \times 4 \rightarrow 216 \times 4) = 14.89$	$28.01 + 37.12(544 \times 4 \rightarrow 1472 \times 4) = 65.13$
Utilization(%)	54	23.4 (새로운 연산 횟수 / latency) $= (87 \times 4 / 1489)$	74.7 (새로운 연산 횟수 / latency) $= (1216 \times 4 / 6513)$

즉, systolic array 크기를 늘리는 경우 latency는 감소하지만, utilization이 급격히 낮아짐을 확인할 수 있고 systolic array 크기를 줄이는 경우 latency는 증가하지만, utilization이 높아짐을 확인할 수 있다.

2-7) Discuss how to improve the latency of my design without changing the clock speed

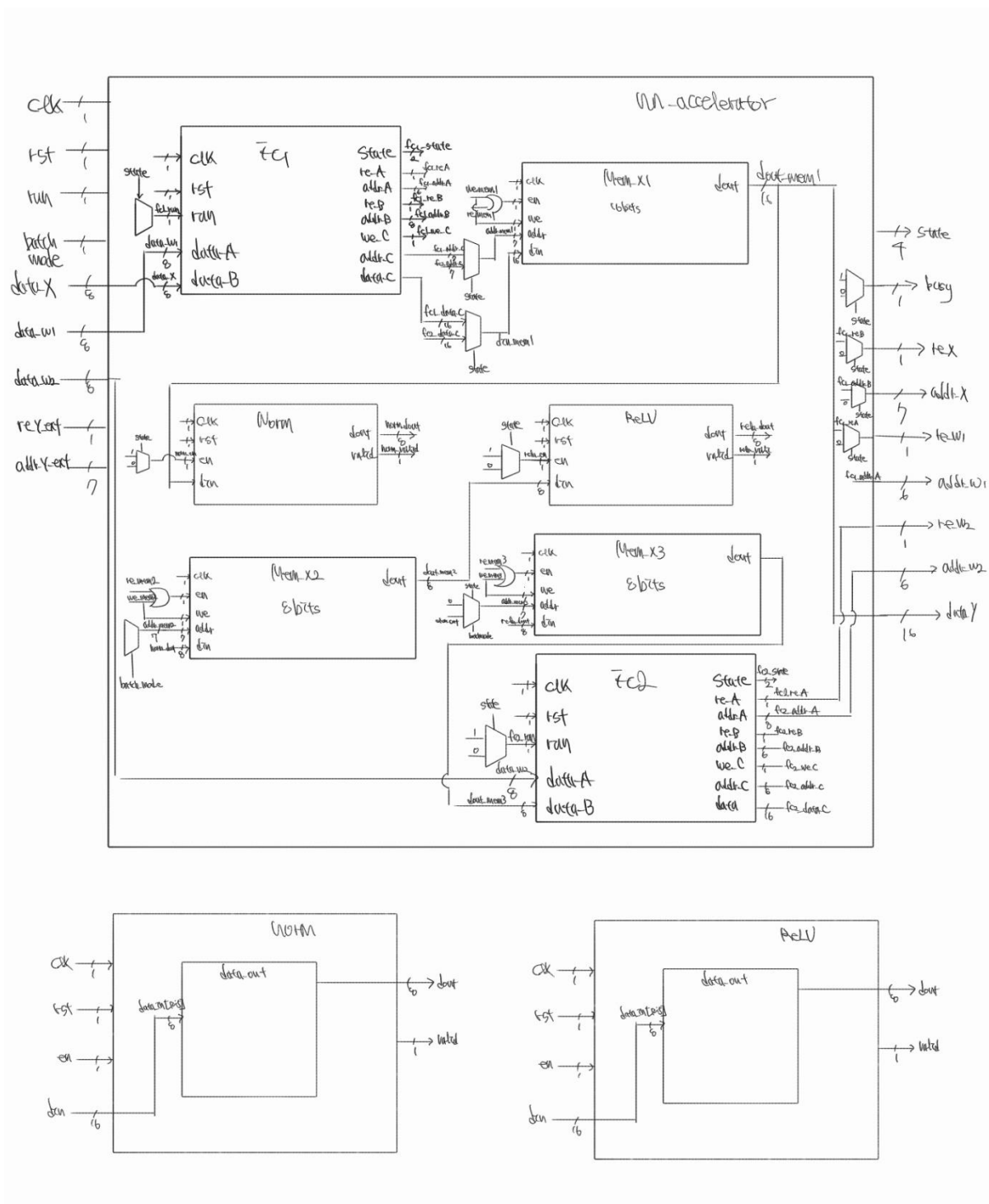
- 현재는 Norm과 ReLU 연산 결과를 모두 메모리 모듈에 저장하고 이를 불러오는 방식으로 다음 연산을 진행한다. 메모리 모듈을 사용하지 않고 내부 버퍼를 활용하거나 즉시 연산을 진행하는 경우 latency를 줄일 수 있을 것이다.
- 각 모듈 연산이 완전히 끝난 뒤 다음 연산을 시작하기 보다, 데이터를 연속적으로 흘려보내 앞 단계의 연산이 끝나는 즉시 다음 단계 연산이 시작되도록 하는 파이프라이닝 기법을 통해 latency를 단축할 수 있다.

2-8) Discuss how to improve the utilization ratio of my design

- 현재 모듈에서는 메모리 입출력에 따라 연산 지연이 커져 utilization이 낮아진다.
- 따라서 데이터 버퍼를 이중화하여 한쪽에서 연산이 이루어지는 동안 다른 버퍼에서 데이터 입출력을 가능하게 한다면 utilization이 높아질 것이다.
- 내부 모듈의 경우 각 연산 모듈마다 처리 시간에서 크게 차이가 발생하기에 병목이 발생하는 모듈의 수를 더 늘리는 방법으로 utilization을 높일 수 있을 것이다.
- 모듈의 연산을 더 세밀한 단위로 나누고, 다른 모듈과의 의존성이 낮은 작업에 대해 병렬성을 추가할 수 있다.

3) Block Diagram

Nn_accelerator, ReLU, Norm module의 block diagram 구성은 다음과 같다.



Nn_accelerator의 인터페이스와 내부 모듈 인터페이스는 위에 작성한 종류, 동작과 동일하다.

2 Simulation Results

1) Display output – 8 Batch

Vivado의 TCI console로 출력한 8 batch 테스트 벤치 실행 결과는 다음과 같다.

8-batch, 16-batch는 테스트벤치 코드에서 TEST_MODE를 0(base) 또는 1(extra)로 설정해 변경할 수 있다.

출력 결과는 batch별 output이며, ref(hex) > out(hex)(decimal) 형태로 출력된다.

Ref(hex)는 디버깅용으로 제공된 실제 출력 결과이며, out(hex)(decimal)은 모듈 통과에 따른 출력 결과이다. 모든 요소에서 정상적으로 진행된 경우 마지막 success가 출력되며 정상 동작을 나타내게 된다.

```
Batch 0 Output [ 0: 7]: ff02 > ff02(-254) 000f > 000f( 15) fec8 > fec8(-312) ff21 > ff21(-223) ff8a > ff8a(-118) 00de > 00de( 222) ff77 > ff77(-137) 0005 > 0005( 5)
Batch 1 Output [ 8: 15]: 00d0 > 00d0( 208) 0010 > 0010( 16) 0076 > 0076( 118) ffdc > ffdc( -36) 0092 > 0092( 146) ff64 > ff64(-156) 010e > 010e( 270) 009a > 009a( 154)
Batch 2 Output [ 16: 23]: 00d6 > 00d6( 214) 0025 > 0025( 37) 002b > 002b( 43) 0040 > 0040( 64) ffce > ffce( -50) ffe5 > ffe5( -27) ffe6 > ffe6( -26) ffad > ffad( -83)
Batch 3 Output [ 24: 31]: fee6 > fee6(-282) ff66 > ff66(-154) ff2c > ff2c(-212) ff56 > ff56(-170) ffda > ffda( -38) feec > feec(-276) ff2c > ff2c(-212) ff6c > ff6c(-148)
Batch 4 Output [ 32: 39]: ff89 > ff89(-119) ffb5 > ffb5( -69) ff3d > ff3d(-195) ff81 > ff81(-127) ff78 > ff78(-136) ffb5 > ffb5( -75) ff4c > ff4c(-180) ff73 > ff73(-141)
Batch 5 Output [ 40: 47]: ff80 > ff80(-128) 0046 > 0046( 70) ff96 > ff96(-106) 001d > 001d( 29) 0050 > 0050( 80) 0060 > 0060( 96) 0135 > 0135( 309) 00e4 > 00e4( 228)
Batch 6 Output [ 48: 55]: 0113 > 0113( 275) 0006 > 0006( 6) 00d9 > 00d9( 217) 009f > 009f( 159) ff6c > ff6c(-148) 001e > 001e( 30) ff11 > ff11(-239) fff6 > fff6( -10)
Batch 7 Output [ 56: 63]: 00ba > 00ba( 186) 006f > 006f( 111) 00ac > 00ac( 172) 0055 > 0055( 85) 001e > 001e( 30) 00c6 > 00c6( 198) 0088 > 0088( 136) 0053 > 0053( 83)
Success!
```

8-batch의 경우 모든 batch, 모든 요소에 대해 정상 출력됨을 확인할 수 있으며 마지막 출력 역시 success로 출력됨을 확인할 수 있다.

2) Display output – 16 Batch

다음은 16-batch 결과를 TCI console로 출력한 결과이다.

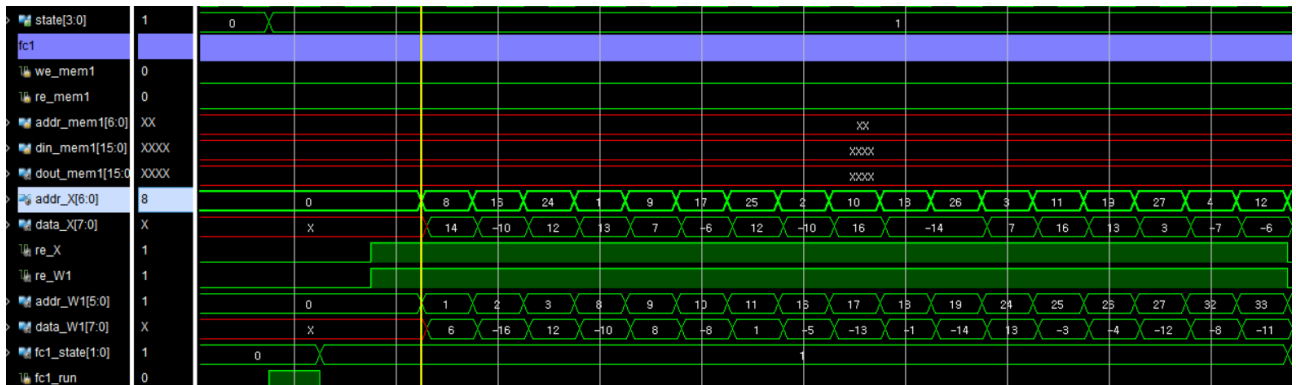
```
Batch 0 Output [ 0: 7]: fef2 > fef2(-270) ff79 > ff79(-135) 00ba > 00ba( 186) ff97 > ff97(-105) fed1 > fed1(-303) ff20 > ff20(-224) 0099 > 0099( 153) 0011 > 0011( 17)
Batch 1 Output [ 8: 15]: fee0 > fee0(-288) fffb > fffb( -5) ffb7 > ffb7( -73) 005b > 005b( 91) ff46 > ff46(-186) 00be > 00be( 190) ffe7 > ffe7( -25) fef3 > fef3(-269)
Batch 2 Output [ 16: 23]: ffb6 > ffb6( -74) 0025 > 0025( 37) ff84 > ff84(-124) ffa2 > ffa2( -94) fff1 > fff1( -15) 0009 > 0009( 9) ff99 > ff99(-103) 0017 > 0017( 23)
Batch 3 Output [ 24: 31]: 00fc > 00fc( 252) ffad > ffad( -83) ff4a > ff4a(-182) 007b > 007b( 123) 0022 > 0022( 34) fffc > fffc( -4) 0019 > 0019( 25) 0061 > 0061( 97)
Batch 4 Output [ 32: 39]: ffd6 > ffd6( -42) 0027 > 0027( 39) ffcf > ffcf( -49) 0021 > 0021( 33) 0053 > 0053( 83) 003e > 003e( 62) fff1 > fff1(-225) 0009 > 0009( 9)
Batch 5 Output [ 40: 47]: ffc1 > ffc1( -63) ff73 > ff73(-141) ff82 > ff82(-126) ff87 > ff87(-121) 002e > 002e( 46) ffa4 > ffa4( -92) fffd > fffd( -3) 00cb > 00cb( 203)
Batch 6 Output [ 48: 55]: 001e > 001e( 30) ff77 > ff77(-137) ff28 > ff28(-216) ffbe > ffbe( -66) fe55 > fe55(-427) fecf > fecf(-305) ff79 > ff79(-135) ffb6 > ffb6(-149)
Batch 7 Output [ 56: 63]: ff8c > ff8c(-116) ffa5 > ffa5( -91) ff86 > ff86(-122) fefb > fefb(-261) ffc8 > ffc8( -56) fe96 > fe96(-362) ffe7 > ffe7( -17) ff47 > ff47(-185)
Batch 8 Output [ 64: 71]: ff4f > ff4f(-177) ffb9 > ffb9( -71) ffc7 > ffc7( -57) ffc9 > ffc9( -55) ff8e > ff8e(-114) ffab > ffab( -85) ff9c > ff9c(-100) ffba > ffba( -70)
Batch 9 Output [ 72: 79]: ffa5 > ffa5( -91) ffb3 > ffb3( -77) ffe6 > ffe6( -26) ff4e > ff4e(-178) ff91 > ff91(-111) ff53 > ff53(-173) ffe5 > ffe5( -27) ffd6 > ffd6(-147)
Batch 10 Output [ 80: 87]: fff0 > fff0( -16) 003a > 003a( 58) 0045 > 0045( 69) 004a > 004a( 74) 0030 > 0030( 48) 002e > 002e( 46) ffea > ffea( -22) 0071 > 0071( 113)
Batch 11 Output [ 88: 95]: 0076 > 0076( 118) ffb7 > ffb7( -73) ffc8 > ffc8( -56) 0092 > 0092( 146) ffc9 > ffc9( -52) 0095 > 0095( 149) 001a > 001a( 26) ffb4 > ffb4( -76)
Batch 12 Output [ 96: 103]: 0004 > 0004( 4) 0083 > 0083( 131) 0013 > 0013( 19) 004e > 004e( 78) 014d > 014d( 333) 00f6 > 00f6( 246) fffd > fffd( -51) ffb3 > ffb3( -77)
Batch 13 Output [ 104: 111]: fefe > fefe(-258) ff57 > ff57(-169) fffb > fffb( -8) 00d6 > 00d6( 214) 0049 > 0049( 73) 00b0 > 00b0( 176) 002c > 002c( 44) 00de > 00de( 222)
Batch 14 Output [ 112: 119]: 001a > 001a( 26) 002e > 002e( 46) 00a1 > 00a1( 161) 003a > 003a( 58) 001b > 001b( 27) fff9 > fff9( -7) 009d > 009d( 157) 0066 > 0066( 102)
Batch 15 Output [ 120: 127]: fff0 > fff0(-112) 0068 > 0068( 104) 001e > 001e( 30) 00c7 > 00c7( 199) 0048 > 0048( 72) 0111 > 0111( 273) 0017 > 0017( 23) 00f1 > 00f1( 241)
Success!
```

16-batch의 경우에도 모든 요소에 대해 정상적으로 출력되며 마지막 success 출력을 통해 정상 동작 여부 역시 확인할 수 있다.

3) Behavioral simulation waveform – 8 batch

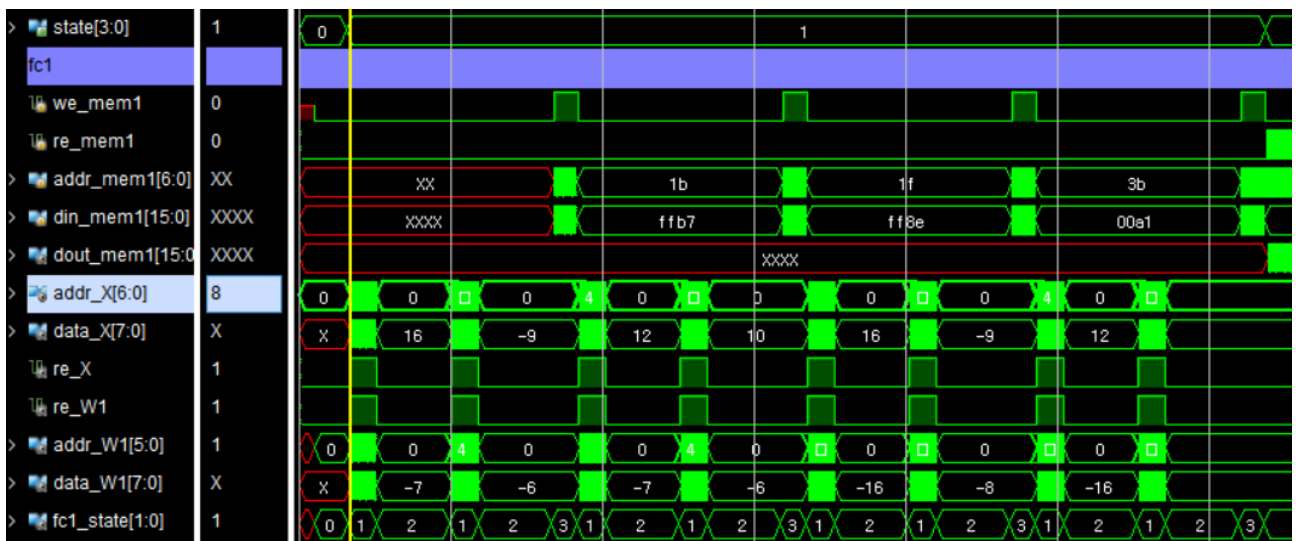
8 batch 기준 state별 동작을 waveform에서 살펴보도록 하자.

3-1) FC1_EXEC STATE



위 waveform은 8-batch 기준, FC1_EXEC state에서 가장 처음 데이터를 불러오는 과정이다. Addr_X와 addr_W1에는 텍스트를 불러오기 위한 주소가 나타나며 data_X와 data_W1에는 addr에 해당하는 데이터들을 호출하도록 한다.

즉, 해당 부분은 8x8 matrix에서 첫 번째 타일에 대한 연산을 진행하기 위한 부분으로 8x8에서 4번의 tiling 진행을 위해서는 위와 같은 파형이 8번은 반복되어야 한다.



위 파형은 8x8 matrix에 대해 8번의 4x4 tiling 연산이 진행됨을 나타낸다.

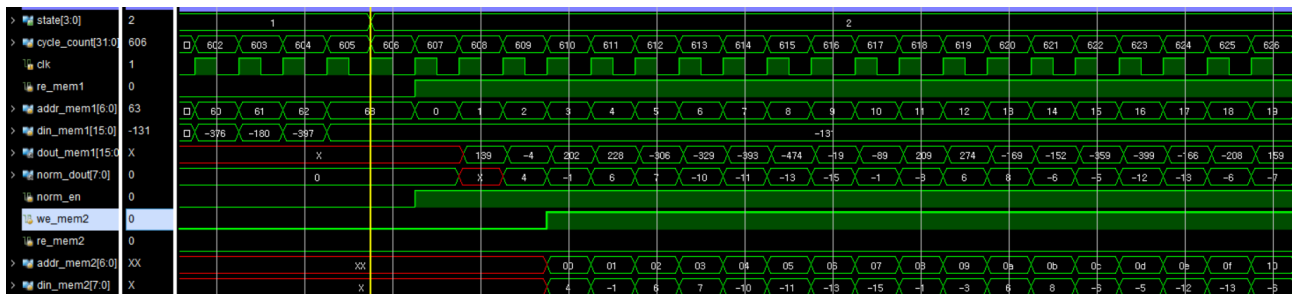
또한, FC1의 연산 결과는 memory x1에 저장하기에 총 4번 we_mem1이 활성화됨을 확인할 수 있고, din_mem1 역시 4번 활성화되어 memory x1에 FC1 연산 결과가 저장됨을 확인할 수 있다.

FC1의 연산 결과가 정확한 위치에 정확한 값이 저장되는 지는 디버깅용으로 제공된 텍스트 파일과 비교하여 확인할 수 있으며 다음 파형을 통해 확인할 수 있다.



Index 0, 1, 2, 3의 경우 파형으로 출력되는 139, -4, 202, 228이 텍스트 파일과 동일하게 출력됨을 확인할 수 있으며 8, 9, 10, 11의 -19, -89, 209, 274 역시 파형과 텍스트 파일의 값이 동일함을 확인할 수 있다.

3-2) NORM_EXEC state

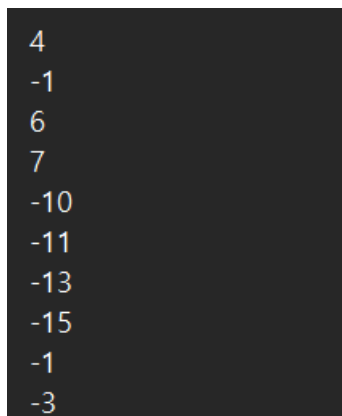


위 파형은 NORM_EXEC에서 데이터가 정상적으로 처리되는 지를 확인하기 위한 waveform이다.

Normalization을 위해서는 memory x1에서 데이터 호출 -> norm 연산 -> memory x2에 데이터 저장 과정을 진행해야 한다.

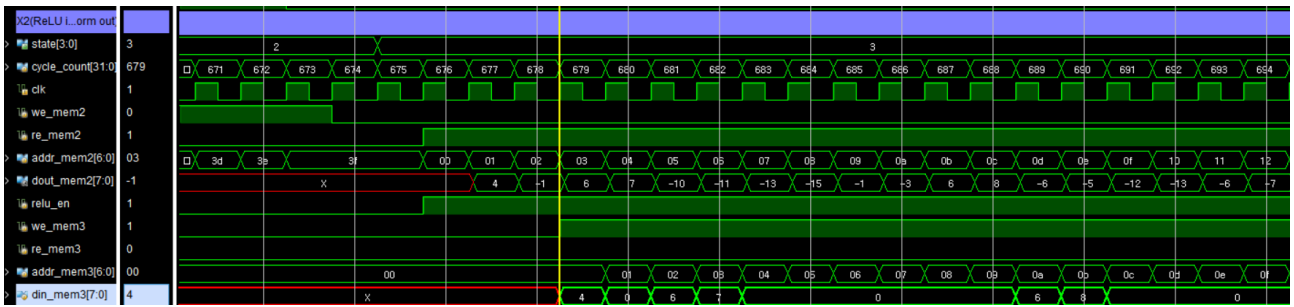
위 waveform에서도 마찬가지로 re_mem1을 활성화시켜 mem1에서 데이터를 불러올 수 있도록 하여 dout_mem1에 address에 맞는 데이터를 호출하고 있으며 norm 연산을 진행하여 해당 데이터를 din_mem2로 전달한다.

이때, we_mem2를 활성화시키며 addr_mem2를 선언하여 데이터를 저장하도록 한다.



위 이미지는 mem2에 저장되어야 하는 normalized 데이터를 0번째 index부터 나타낸 디버깅용 텍스트 파일의 일부로, waveform에서 4, -1, 6, 7, ... 순서로 mem2에 저장하는 din_mem2 데이터와 디버깅용 데이터가 동일함을 확인할 수 있다.

3-3) RELU_EXEC state



위 파형은 RELU_EXEC에서 데이터가 정상적으로 처리되는 지를 확인하기 위한 waveform이다.

Activation을 위해서는 memory x2에서 데이터 호출 -> relu 연산 -> memory x3에 데이터 저장 과정을 진행해야 한다.

위 waveform에서도 마찬가지로 re_mem2을 활성화시켜 mem2에서 데이터를 불러올 수 있도록 하여 dout_mem2에 address에 맞는 데이터를 호출하고 있으며 relu 연산을 진행하여 해당 데이터를 din_mem3로 전달한다.

이때, we_mem3를 활성화시키며 addr_mem3를 선언하여 데이터를 저장하도록 한다.



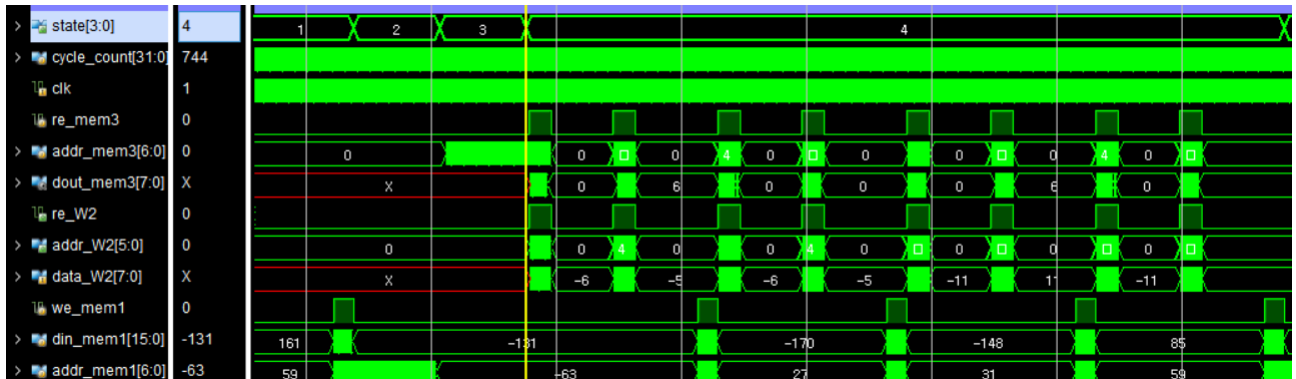
위 이미지는 mem3에 저장되어야 하는 activation 데이터를 0번째 index부터 나타낸 디버깅용 텍스트 파일의 일부로, waveform에서 4, 0, 6, 7, 0... 순서로 mem3에 저장하는 din_mem3 데이터와 디버깅용 데이터가 동일함을 확인할 수 있다.

3-4) FC2_EXEC STATE

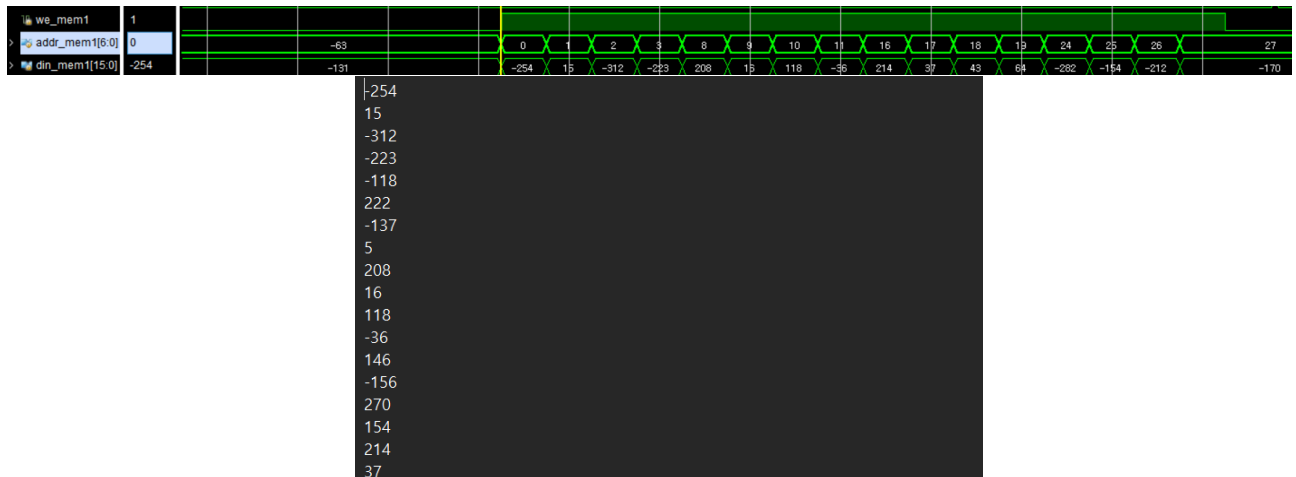
FC2에서도 FC1와 마찬가지로 총 8번의 연산이 진행되며 그 중 2번 간격으로 총 4번은 FC의 결과를 저장하도록 한다.

FC2의 진행 과정은 다음과 같다.

- re_mem3를 활성화시키며 addr_mem3를 전달하여 data_mem3 호출
- re_W2를 활성화시키며 addr_W2를 전달하여 텍스트 파일로부터 data_W2 출력
- data_mem3와 data_W2에 대해 FC 연산을 진행하여 mem1에 다시 저장
- mem1에 저장을 위해 we_mem1 활성화 및 addr_mem1 전달



위 파형은 결과 역시 위에서 정의한 순서를 만족함을 확인할 수 있다. 총 8번 연산을 위해 mem3와 w2로부터 데이터를 8번 불러오며 4번의 연산 결과를 mem1에 저장한다.



위 파형은 실제로 mem1에 저장되는 최종 결과와 addr을 나타낸 것으로, index가 0, 1, 2, 3인 경우 -254, 15, -312, -223 data가 mem1에 저장되며 index가 8, 9, 10, 11인 경우 208, 16, 118, -36 data가 mem1에 저장됨을 파형에서 확인할 수 있다.

또한 해당 data들은 위에 첨부한 y의 실제 출력 값과 동일함을 확인할 수 있다.

4) Behavioral simulation waveform – 16 batch

4-1) 16 batch

16 batch의 경우 8 batch와 달리 W1, W2의 크기는 유지한 채로 input 데이터인 X가 16x8 형태로 변경된다. 따라서 기존 FC1 -> Norm -> ReLU -> FC2의 FSM 진행을 2번 반복하여 16x8 데이터에 대해 연산을 진행하도록 한다.

아래 파형은 8 batch와 달리 FSM 진행이 2번 반복됨을 나타낸다.

